BABEL: a Novel Software Vulnerability Detection Framework for Breaking Language Barriers

Xia Li, Yuhang Lin*, Yongqiang Zheng, Junyi He, Rihu Liao, Junlang Wang
School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China
{xiali, yuhanglin, yqzheng, junyihe, liaorihu, junlangwang}@gdufs.edu.cn

2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3503-9568-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSME58944.2024.00060

***Abstract*—Software vulnerability detection aims to identify latent weaknesses in source code to protect software systems from attacks. Most of the current neural network-based approaches rely on preprocessing source codes using code parsers to obtain program dependency graphs for downstream vulnerability detection tasks. However, as source codes may contain incorrect format and special functions, this paradigm may cause parsing errors due to their failure to satisfy the parser's presets or requirements. In addition, modern software development often involves the integration of multiple programming languages, presenting challenges for conventional code parsers that are designed for specific programming languages. To address these issues, we present a novel detection framework that can identify software vulnerabilities in a programming language-agnostic manner without relying on code parsers. We conduct extensive experiments on the CodeXGLUE, Reveal, and FUNDED public datasets and evaluate them against leading vulnerability detection benchmarks. Experimental results show that BABEL sets new performance standards, demonstrating superior accuracy and F1 scores across various tasks, including those in language-independent settings, demonstrating the applicability of BABEL in real-world application scenarios. Our code is available at https://github.com/gdufsnlp/BABEL**

***Index Terms*—Software vulnerability detection, hierarchical attention network, graph neural network, code embedding.**

I. Introduction

Software vulnerability detection aims to identify potential weaknesses in software systems that may cause the software to operate beyond its intended design, exposing attackers to exploit it. It is an important task that is widely used in the field of cyber security. According to Check Point Research [1], global cyberattacks climbed by 8% in the second quarter of 2023, the highest increase in two years. This development highlights the critical need for enhanced vulnerability detection mechanisms.

Early vulnerability detection approaches mainly focused on the extraction of features and patterns crafted by human experts and used statistical learning methods to detect and predict vulnerabilities [1]–[5]. Although these approaches are effective, they also face challenges in keeping pace with the diverse range of vulnerabilities, varying by type and library. With the rise of deep learning, many studies based on neural networks have been proposed and achieved promising results [6]–[15]. These methods automatically learn the semantics and program structure of the source code by integrating different neural network frameworks to effectively improve the performance of vulnerability detection.

[1]https://www.checkpoint.com/

Current neural network-based methods can be generally grouped into two types: token-based methods and graph-based methods. Token-based methods treat source codes as flat token sequences and use natural language processing methods to encode and extract source code semantics [7], [8], [16]–[18]. Despite their simplicity and efficiency, these methods struggle to precisely capture a program's internal architecture, including data flow and control flow. Recent studies [10], [19]–[22] employ Graph Neural Networks (GNNs) to acquire a more comprehensive representation of programs by capturing both semantic and structural dependencies within source code. These approaches use code analyzers (e.g., Joern [23]) to convert source code into a structured format that neural networks can analyze, such as Program Dependency Graphs (PDGs). Subsequently, GNNs are employed to learn the complex interdependencies in the PDGs, encompassing data and control flow information.

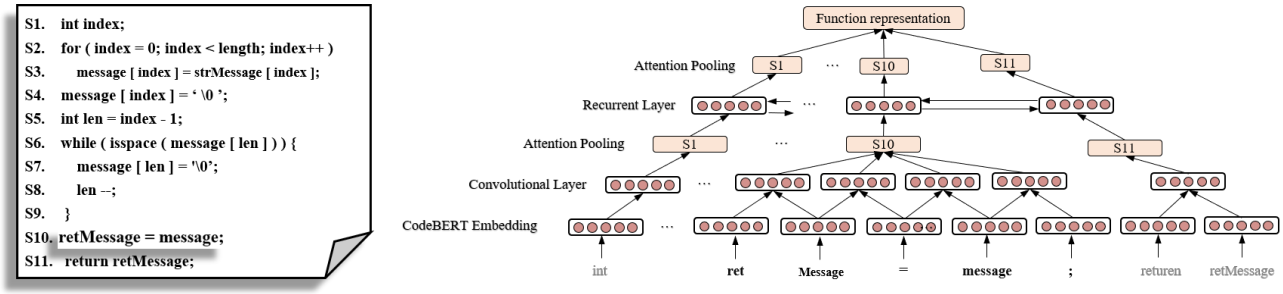
Despite the prospective efficacy of the above methods, we argue that there are still the following limitations:

1) Most existing neural network-based approaches rely on preprocessing source codes with code parsers to obtain program dependency graphs before proceeding to downstream vulnerability detection tasks. However, some source codes may contain incorrect format and special functions (such as enclosing the function definition within a macro), which will cause parsing errors due to their failure to satisfy the parser's presets or requirements. These parsing errors may reduce the amount of valid data in the dataset, thereby introducing bias into the models trained on the cleansed datasets. For instance, Yuan et al. [24] had to eliminate functions that Joern parser was unable to handle, resulting in a decrease in their FFMpeg+Qemu and Chrome+Devign datasets from 27,318 to 25,524 and 22,734 to 21,059, respectively.

2) Modern software development often involves the integration of multiple programming languages, posing a challenge for conventional code parsers that are designed for specific programming languages. This can lead to deep learning methods that rely on parsers performing well in some languages but not in others. As an illustration, the Joern [23] parser can handle C and Java codes but cannot parse PHP codes. This limitation is particularly challenging in application scenarios where web applications are developed in both PHP and Java. To ensure the application's complete security, it is necessary to include security protection in both programming languages. Moreover, employing different parsers for various program-

2576-3148/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSME58944.2024.00060

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.



(a) A source code segment

(b) A partial hierarchical attention network

Fig. 1. An explanatory example illustrates how than encodes a piece of source code from token level to code line level and finally to function level. The code text is first passed through the embedding layer of a pre-trained language model to obtain the token's embedding. Then, we apply a one-dimensional convolutional layer on top of the pre-trained embedding to obtain the n-gram representation, followed by attention pooling to capture the code line-level representation. We apply LSTM to the line-of-code vectors in the recurrent layer and then pass another attention-pooling layer to obtain the overall function-level representation.

ming languages to train new models would not exhibit efficient scalability but instead create unnecessary computational costs and delays.

Then the question comes naturally: can we develop a method that is independent of code parsers as well as programming languages, yet can acquire semantic and structural information about program source codes, such as data flow and control flow? To the best of our knowledge, only a few previous studies have investigated this particular problem. ReGVD [25] is one of them. It treats source code as a flat sequence of tokens, creates a relational graph using a sliding window between tokens, and employs a residual graph neural network to learn representations of codes. However, its dependence on sliding windows and flat token sequences inadequately captures long-range dependencies such as control flow relationships between lines of code, reducing its effectiveness in identifying complex or subtle security vulnerabilities.

To address the above challenges, in this paper, we present **BABEL**, a novel scala**B**le software vulner**AB**ility detection framework with hi**E**rarchical attention and program f**L**ow analysis. Our framework does not rely on specific code parsers and can accurately represent the fundamental structures and interdependencies of programs in a language-agnostic manner. To be precise, we start by assigning a representation to each program token using pre-trained language model embeddings. These embeddings have been extensively trained on various language datasets, making our model capable of identifying vulnerabilities in several programming languages. We then employ an hierarchical attention network (HAN) to model the structural information of the source code and accurately capture its detailed semantic and architectural connections. Fig.1 demonstrates the process of encoding each code line using convolutional neural networks (CNNs [26]) and an attention mechanism. This approach allows us to collect tokens that are semantically connected and obtain a representation at the line level. Next, the code lines of representations were fed into LSTM [27], and the attention mechanism was employed to capture the relationships between code lines in order to acquire the function-level representation of the source code. In addition, in order to model the internal dependencies of the program, we further propose a heuristic program dependency graph construction strategy to build the data dependency graph and control dependency graph. By doing so, the model can identify the critical execution path for detecting vulnerabilities in various programming languages.

We conducted extensive experiments on the CodeXGLUE [28], Reveal [10], and FUNDED [13] datasets, which are widely used benchmarks for detecting public vulnerability. Experimental results demonstrate that BABEL surpasses all other state-of-the-art software vulnerability detection approaches. In common vulnerability detection tasks, BABEL achieves the best accuracy and F1 scores. Notably, in vulnerability detection scenarios that are not dependent on language, BABEL scores a remarkable 15.02% increase in the F1 score measure compared to the top-performing baseline model. To summarize, this work's contributions are as follows:

(1) We employ a pre-trained language model and a hierarchical attention network to encode the source code in order to effectively express the semantics of various programming languages and fully capture the information of the code at both the line-level and function-level.

(2) We present two novel strategies to improve the comprehension of program logic in a way that is not dependent on any specific programming language. These strategies effectively integrate data and control dependencies in program source codes and enhance linkage and interaction analysis among distinct lines of code.

(3) To the best of our knowledge, we are the first to integrate the hierarchical structure of the code (using HAN) and the data and control dependencies (using GNN) for detecting vulnerabilities that are not specific to programming languages. The comprehensive empirical findings illustrate the excellent performance of our model as well as its practical value in real-life scenarios.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

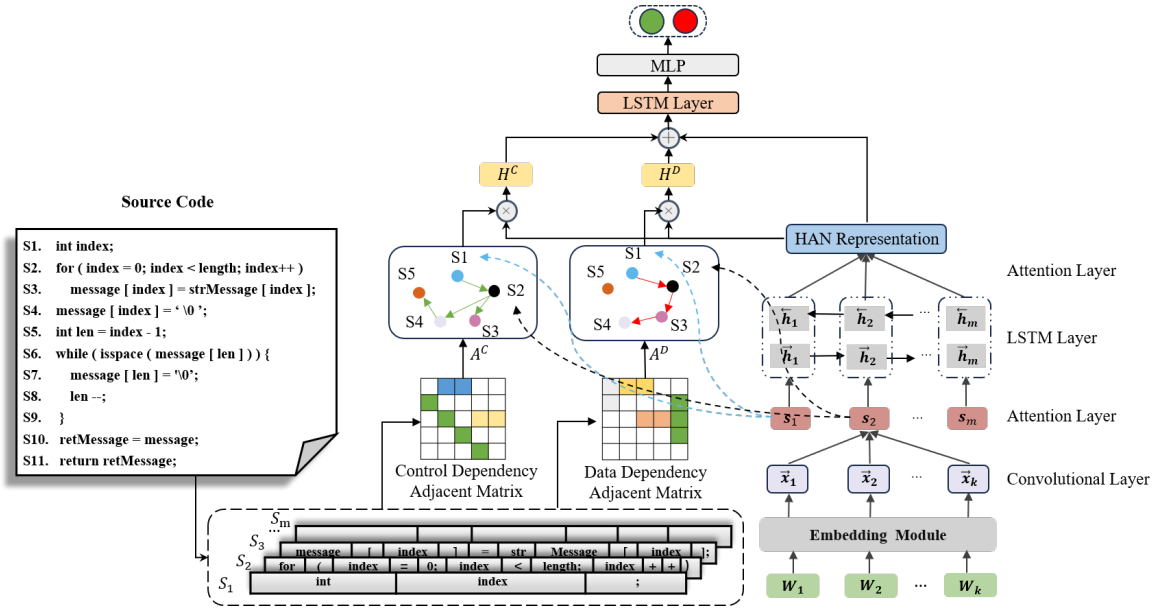


Fig. 2. Overview of the BABEL model. Given a code snippet, from left to right, BABEL generates a control dependency adjacency matrix and a data dependency adjacency matrix using heuristic program dependency graph construction on the lines of code. The matrix will be used as input to the graph neural network in the next step. On the right side of the image, BABEL encodes the source code using a hierarchical attention network, first encoding the token in each line of code using a CNN and the attention mechanism, and then LSTM and attention mechanism are used to learn the dependencies between lines in the program. Finally, the HAN representation and the GNN representation are integrated by the attention mechanism and the prediction results are obtained.

II. Proposed Model

A. Problem Definition

Given a collection of examples D=$\{(c_i, y_i) \mid c_i \in \mathcal{C}, y_i \in \mathcal{Y}\}_{i=1}^{n}$, where $\mathcal{C}$ denotes the set of raw source codes, c_i represents the i-th source code in the dataset, $\mathcal{Y} = \{0, 1\}$ represents the binary label set (1 indicating vulnerability and 0 otherwise), y_i represents the label of the $i-th$ source code, and n is the total number of instances. Our model aims to predict the vulnerability label y_i for the $i-th$ source code c_i in the dataset.

B. Overview of BABEL

To fully obtain rich semantics and the data and control dependencies in the source code in a language-independent manner, we introduce a novel source code vulnerability detection system, namely BABEL. As shown in Figure. 2, BABEL consists of three main modules: Code Encoding, Program Dependency Graph Construction, and Feature Fusion.

Code Encoding. Regarding the hierarchical nature of the source code, we encode the source code using HAN, as it mirrors the inherent structure of code—ranging from tokens to lines and, subsequently, to entire functions or programs. This layered approach allows the model to learn rich and nuanced representations that capture both syntactic and semantic elements of the code.

Program Dependency Graph Construction. We regard a line of code as a node and perform heuristic rules to build relationships between nodes. These heuristic rules focus on both data dependencies and control dependencies of the code, thus extracting the rich program dependencies within the source code.

Feature Fusion. Feature fusion is to form an integrated representation that merges structural insights from HAN and the dependency analysis of GNN. This synergy provides a comprehensive view of understanding the code.

In the following sections, we will describe Code Encoding, Graph Generation, and Feature Fusion in detail.

C. Code Encoding

Our approach involves using an attention-based recurrent neural network with convolutional features as the foundational model, similar to the one referenced in [29]. As shown in Figure.2, our approach involves transforming function-level source code into a structured representation, leveraging Hierarchical Attention Networks (HAN) to address the inherent complexity typical of source code effectively.

To address the challenges of language-specific vulnerability detection, our method instead of using traditional embedding methods like Word2Vec [30] and GloVe [31], which primarily treat code as natural language and usually train on a single-language corpus, we prefer pre-train language model(PLM) embeddings to provide a more generalized representation, which is crucial for identifying vulnerabilities across various environment without bias towards specific language.

Code-Line Representation: Given a sequence of k words of code line $s_t = \{w_1, w_2, ..., w_k\}$, we first use x_i to denote

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

the CodeBERT [32] embedding of each word w_i. Subsequently, a 1D convolutional layer [26] processes these embeddings to derive n-gram features, followed by an attention mechanism that extracts codeline-level insights. Also to ensure fairness of the comparison, we initialize the node feature vectors using only the token embedding layer of the pre-trained programming language model, rather than fine-tuning the entire model.:

$$\boldsymbol{c_i} = \text{CNN}([\boldsymbol{x_i} : \boldsymbol{x_{i+l-1}}]),\ i = 1, 2, \cdots, k \tag{1}$$

$$\boldsymbol{h_t} = \text{Attention}([\boldsymbol{c_1} : \boldsymbol{c_k}]) \tag{2}$$

where l is the kernel size of CNN, $\boldsymbol{c_i}$ is the output of the convolution operation applied to *i*-th to the *i+l-1* th CodeBERT embedding, and $\boldsymbol{h_t}$ is the representation of code line s_t.

Function Representation: After obtaining the representations of the code lines $\boldsymbol{h_1}$,$\boldsymbol{h_2}$,...,$\boldsymbol{h_m}$, where $\boldsymbol{m}$ is the number of code lines in the source code. The final function representation is achieved by feeding these line representations into a Long Short-Term Memory (LSTM) [27] networks and applying another layer of multi-head self-attention pooling, which leads to a total understanding of the function's semantic value.

$$\boldsymbol{r_t} = \text{LSTM}(\boldsymbol{h_{t-1}}, \boldsymbol{h_t}), t = 1, 2, \cdots, m \tag{3}$$

$$\boldsymbol{H^f} = \text{MultiHeadAttention}(\boldsymbol{r_t}, \boldsymbol{r_t}, \boldsymbol{r_t}) \tag{4}$$

where $\boldsymbol{r_t}$ is the output of LSTM at the *t*-th time step, and $\boldsymbol{H^f}$ is the final function-level code representation. We use the multi-head self attention mechanism which is the same as the Transformer [33] model to capture the dependencies in the function level of code.

D. Program Dependency Graph Construction

Language-independent program dependency modeling for vulnerability detection is a challenging goal that requires a trade-off between practicality and generality. To achieve this goal, we propose two heuristic rules: the symbolic data dependency graph (DDG) construction and the indentation-based control dependency graph (CDG) construction to accurately capture the rich semantics and internal dependencies of code text in a language-independent manner. Our heuristic design principles are based on generic programming constructs that make minimal linguistic assumptions about the syntax of a programming language. For example, common programming constructs such as variables, functions, control flow statements (e.g., if-else, loops), and indentation levels are widely available in most programming languages (e.g., Java, Swift, PHP, Python, Ruby, etc.). In the following sections, we will provide a complete code preprocessing procedure and explain these two rules in detail.

1) Source code standardization.: We note that different codebases have different coding styles. In order to make our vulnerability detection approach applicable to various programming languages and projects, our solution is to ensure consistency of coding specifications during the training and inference phases so that the code formatting style is consistent

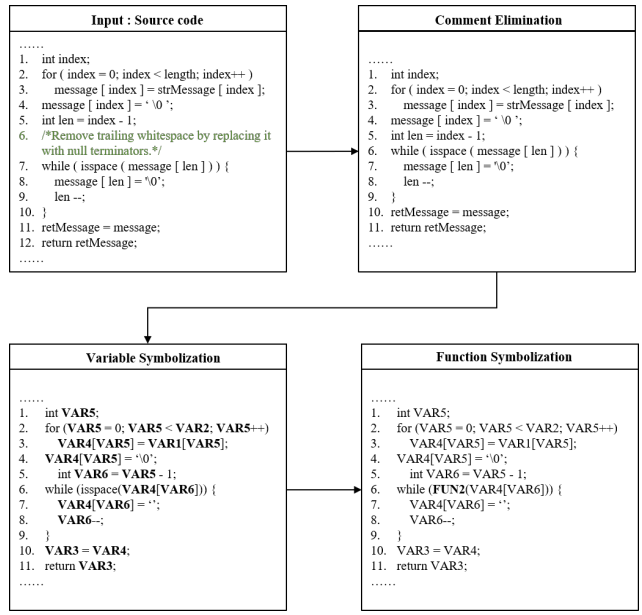


Fig. 3. Steps for source code normalization. The three steps of removing comments and symbolizing variable and function names serve as standard input for constructing program dependency graphs.

across languages. We normalize the source code according to the following steps:

Step 1: Comment Elimination. We strip all comments from the source code, recognizing that while they provide human context, they are extraneous for the semantic analysis of the program's functionality.

Step 2: Uniform formatting. We used code formatting tools (e.g., clang-format[2]) to keep the processed code uniformly standardized while retaining as many statements per line as possible. This facilitates our subsequent heuristic-based rule extraction of dependencies between lines, especially to identify the most important lines of statements in the code, which is crucial for the vulnerability detection task.

Step 3: Variable and Function Symbolization We systematically rename user-defined variables to a uniform symbolic nomenclature (e.g., $\mathbf{VAR1}, \mathbf{VAR2}$) and also standardize function names to symbolic identifiers (e.g., $\mathbf{FUN1}, \mathbf{FUN2}$). Notably, this normalized code is only used for subsequent program dependency extraction, and we retain the original variable and function names as inputs to the model coding.

2) Data Dependency Graph Construct.: Data dependency analysis allows us to gain a deeper understanding of how information moves through a program. Specifically, we consider each line of code as a node of the data dependency graph. We then construct the data dependency graph according to the following definition.

Given a program source code P which denoted as $P = \{s_1, s_2, \ldots, s_m\}$, where s_i is a code line in the source code, $i = 1, 2, ..., m$, we name s_i is a statement. Within these

[2]https://clang.llvm.org/

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

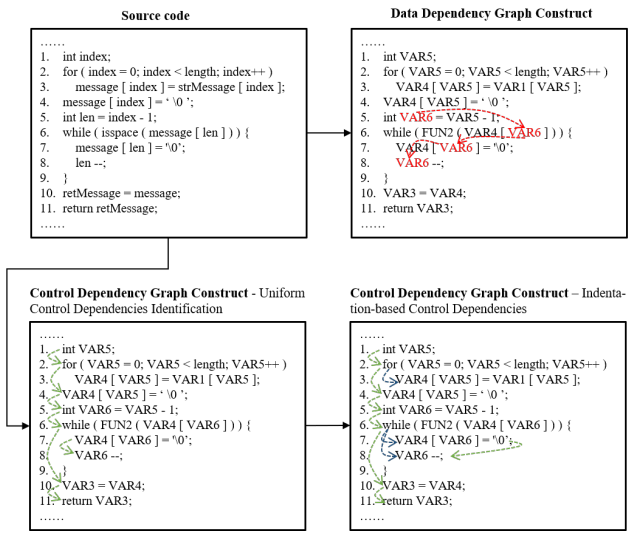


Fig. 4. Language-Independent Construction of Program Dependency Graph. The standardized source code is used to construct a data dependency graph by the flow of data and a control dependency graph by indentation analysis between lines of code.

statements, variables $v_{i,j}$ and function identifiers $f_{i,j}$ are marked. A dependency exists if a variable or function defined or used in one statement s_i is used in another s_u. After that, we concatenate all uses of the same variable or function in different lines to capture the dependencies between variables and functions in those lines. A data dependency adjacency matrix $\mathbf{A^D} \in \mathbb{R}^{n \times n}$ of the graph of the function-level code is derived as follows:

$$A^D_{i,j} = \begin{cases} 1, & \text{if } s_i, s_j \text{ contains variable dependency} \\ 1, & \text{if } s_i, s_j \text{ contains function dependency} \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

3) Control Dependency Graph Construct.: To understand how different lines of a program influence each other's execution, we build the control dependency graph according to the following definition. Given two distinct statements s_i and s_j (where $i \neq j$) in a program source code $P = \{s_1, s_2, \ldots, s_m\}$, if the execution outcome of s_i influences the execution of s_j, then s_j is considered to be control-dependent on s_i, as a principle for modeling control dependencies. This graph construction is carried out through two primary steps:

Step 1: Uniform Control Flow Identification: Lines of code with the same indentation level typically represent a series of statements that are logically executed in sequence, as shown in Figure.4, we join lines of code with the same indentation level to represent the natural progression of control dependencies. This step reflects the way the control flow graph displays all executable paths sequentially.

Step 2: Indentation-based Control Dependencies: Then we build control dependencies based on a hierarchy of indentations. Because the execution of a statement with a greater degree of indentation is subject to judgment based on the result of the execution of its previous code statement. For example, statements nested within an "if" block which implies a greater degree of indentation are considered to be dependent on the "if" statement itself. Because nested statements need to be conditionally executed based on the result of the "if" statement (e.g., blue part in 4 Line 2, Line 3). By following the above steps, we can construct a control dependency graph both by adhering well to the definitions and by understanding how different lines of the program affect each other's execution. A control dependency adjacency matrix $\mathbf{A^C} \in \mathbb{R}^{n \times n}$ of the graph of the function-level code is derived as follows:

$$A^C_{i,j} = \begin{cases} 1, & \text{if } s_i, s_j \text{ contains uniform control dependency} \\ 1, & \text{if } s_i, s_j \text{ contains indentation-based control} \\ & \text{dependency} \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

E. Feature Fusion

This feature fusion module ensures that our model does not merely interpret the code as a simple sequence of tokens or a mere network of function calls. Instead, it utilizes HAN's hierarchical structural representation and the detailed data/control dependency insights from GNNs for more effective and nuanced vulnerability detection.

To obtain the data dependency representation, we input the data dependency adjacency matrix into the GCN to learn the data dependencies of a given function.

$$\boldsymbol{h_i^{D,l}} = \text{ReLU}(\tilde{A}^D_i \boldsymbol{h_i^{l-1}} \boldsymbol{W^{D,l}} + \boldsymbol{b^{D,l}}) \tag{7}$$

$$\tilde{A}^D_i = A^D_i / (E^D_i + 1) \tag{8}$$

where $\boldsymbol{h_i^{D,l-1}}$ is the hidden representation obtained from the previous GCN layer, $E^D_i = \sum_{j=1}^{n} A^D_{i,j}$ is the degree of A^D_i. The initialization of nodes in GCN is to use code line representation $\boldsymbol{h_t}$ in Equation 2. Similarly, we use the control dependency adjacency matrix $A^C_{i,j}$ to learn the control flow representation through another GCN.

Then the integration of features from HAN and GCN is achieved through an attention mechanism:

$$\boldsymbol{Q} = \text{SUMPOOLING}(\boldsymbol{H^f}) \tag{9}$$

$$\boldsymbol{K_D} = \{\boldsymbol{h_1^D}, \boldsymbol{h_2^D}, ..., \boldsymbol{h_m^D}\}, \boldsymbol{K_C} = \{\boldsymbol{h_1^C}, \boldsymbol{h_2^C}, ..., \boldsymbol{h_m^C}\} \tag{10}$$

$$\alpha = \text{Softmax}(\boldsymbol{Q}\boldsymbol{K_D}^{\boldsymbol{T}}), \beta = \text{Softmax}(\boldsymbol{Q}\boldsymbol{K_C}^{\boldsymbol{T}}) \tag{11}$$

$$\boldsymbol{H^D} = \alpha \cdot \boldsymbol{K_D}, \boldsymbol{H^C} = \beta \cdot \boldsymbol{K_C} \tag{12}$$

where $\boldsymbol{Q}$ is obtained by applying a sum pooling to HAN representation. $\boldsymbol{K_D}$ and $\boldsymbol{K_C}$ are the GCN outputs of data dependency graph and control dependency graph. We use soft attention to obtain the data-aware representations $\boldsymbol{H^D}$ and the control-aware representations $\boldsymbol{H^C}$.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

F. Model Training

The final classification features are concatenated by the original HAN representation $\boldsymbol{H^f}$ and the data-aware and control-aware representations $\boldsymbol{H^D}$,$\boldsymbol{H^C}$.

$$\boldsymbol{H_{fusion}} = \text{Concat}(\boldsymbol{H^f}, \boldsymbol{H^D}, \boldsymbol{H^C}) \tag{13}$$

Here, the hidden representations of individual code lines interact with the max pooling layer via Bidirectional LSTM, which utilizes the final representation as input.

$$\boldsymbol{H_{out}} = \text{Bi-LSTM}(\boldsymbol{H_{fusion}}) \tag{14}$$

$$\boldsymbol{z} = \text{MAXPOOLING}(\boldsymbol{H_{out}}) \tag{15}$$

It processes the output through a fully connected layer followed by a sigmoid activation, mapping it to the probabilities of vulnerability results.

$$\boldsymbol{y_i} = \text{Sigmoid}(\text{MLP}(\boldsymbol{z}) \tag{16}$$

We use the cross-entropy as the training objective loss:

$$\mathcal{L} = -\sum_{i=1}^{s} \hat{y}_i^j \log y_i^j \tag{17}$$

where s is the number of training samples, y_i is the correct distribution of vulnerability.

III. Evaluation

A. Research Questions

Our goal is to develop a novel framework for efficient vulnerability detection that departs from traditional code parsers and runs independently of the programming language. To evaluate the effectiveness of our model, we ask the following three questions:

RQ1: How effective is BABEL at detecting vulnerabilities?

RQ2: What is the detection performance of BABEL when detecting across program language vulnerability?

RQ3: Does BABEL understand the nature of vulnerabilities in different languages?

RQ4: How does HAN, Data-dependency, and Control-dependency contribute to the performance of BABEL?

B. Dataset

We evaluate BABEL using three datasets: CodeXGLUE, Reveal, and FUNDED, which can be seen in Table 1, focusing on the programming languages C, C++, PHP, and Java. We utilize the CodeXGLUE dataset for function-level vulnerability detection, which was first proposed by Zhou et al [19] and later re-packaged in CodeXGLUE [28]. This balanced dataset is composed of function-level examples collected from the well-known open-source projects QEMU and FFmpeg. The Reveal dataset, built by Chakraborty et al [10]. with data from the Chromium project and the Linux Debian kernel, has a 1:10 ratio of vulnerable programs to non-vulnerable programs, providing a realistic challenge for identifying vulnerabilities. For assessing BABEL's proficiency across varied programming languages, we incorporate Java samples from the FUNDED dataset which was developed by integrating resources from SARD[3], NVD[4], and an array of open-source GitHub[5] projects.

TABLE I
STATISTICS OF THREE PUBLIC DATASETS.

Dataset	Samples	Languages	Ratio (Vul: Non-vul)
CodeXGLUE [28]	27,318	C/C++	1:1.2
Reveal [10]	18,169	C/C++	1:9.9
Funded [13]	167,496	JAVA	1:2.7
Funded [13]	37,808	PHP	1:1.2

C. Baseline Methods

In this paper, we compare with some baselines as follows. We use FlawFinder [34] and CPPcheck [35] as baselines for they are widely used static detection tools. We also compare Reveal [10], Devign [19], LineVD [36], ReGVD [25], and CSGVD [37] because these graph-based approaches are very popular for accurate vulnerability detection by analyzing program dependency graphs. Due to the advancement demonstrated by existing pre-trained language models, we also compared CodeBERT [32], GraphCodeBERT [38], CoTexT [39], PLBART [40], C-BERT [41], and LineVul [42]. There are also sequence-based models such as Russell et al [7].

D. Implementation details

The experiments are conducted on an Ubuntu 20.04 server with an NVIDIA GeForce GTX 3060 GPU and an Intel CPU with 512 GB memory. The neural network is trained with a batch size of 64. The number of GCN layers is set to 2 and the head of meta-path attention is set to h = 8. We adopt Adam optimizer to train our model with a learning rate of 5e-4. The embedding dimension is set to 768. We use gradient descent to train the neural network with a learning rate of 0.0005 for a maximum number of training epochs with 100 which the patience is 20. Additionally, we applied a dropout strategy with a ratio of 0.5. To ensure the fairness of the experiments, we use the same data partitioning in keeping with the previous approach [10], [13], [28]. We randomly divide the dataset into disjoint training, validation, and testing sets in a ratio of 8:1:1. As in previous work [20], [37], [43], we use accuracy, precision, recall, and F1 score as metrics for evaluating the model.

IV. RESULTS

A. RQ1: How effective is BABEL at detecting vulnerabilities?

We compare our model with classical static vulnerability detection tools and SOTA baselines based on different networks. As shown in TableII, BABEL achieves better results on the imbalanced dataset Reveal and outperforms two traditional static tools and all mentioned token-based and graph-based

[3]https://samate.nist.gov/SARD/
[4]https://nvd.nist.gov/
[5]https://github.com/

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

TABLE II
VULNERABILITY DETECTION RESULT SUMMARY

Method	Model	Accuracy (%)	F1 Score
Static Tools	FlawFinder [34]	81.67	18.18
	CPPcheck [35]	83.71	3.27
Token-based	Russell et al. [7]	89.07	19.80
	CodeBERT [32]	90.19	40.53
	GraphCodeBERT [38]	90.98	33.22
	CoTexT [39]	90.50	32.50
	PLBART [40]	90.54	32.60
	LineVul [42]	91.25	42.98
Graph-based	Reveal [10]	84.37	41.25
	Devign [19]	88.11	26.85
	LineVD [36]	89.75	43.31
	ReGVD [25]	89.93	17.92
	CSGVD [37]	89.37	43.68
Ours	**BABEL**	**91.60**	**44.31**

TABLE III
BENCHMARK EVALUATION RESULTS

Rank	Model	Data Size	Param size	Acc. (%)
1	CoTexT [39]	375M	220M	66.62
2	C-BERT [17]	8.1M	110M	65.45
3	**BABEL (ours)**	**8.5M**	**21M**	**65.06**
4	VulBERTa-MLP [44]	2.28M	125M	64.75
5	PLBART [40]	680M	140M	63.18
6	Code2vec [17]	-	-	62.48
7	CodeBERT [32]	8.5M	125M	62.08
8	RoBERTa [45]	2.4M	125M	61.05
9	TextCNN [16]	-	-	60.69
10	BiLSTM [27]	-	-	59.37

methods. Compared to the classic and widely used static vulnerability detection tools FlawFinder [34] and CPPcheck [35] for comparison. BABEL demonstrates a leading performance with an accuracy of 91.60% and an F1 score of 44.31% which significantly outperforms them on both metrics. This shows that BABEL has made significant progress in detecting vulnerabilities more accurately and balancing precision and recall. The improved performance of deep learning models like BABEL is attributed to their ability to continually learn and adapt to new vulnerabilities, unlike traditional static tools [34], [35] which depend on manual updates for new threats.

We then compare BABEL with token-based deep learning methods and demonstrate that BABEL outperforms these methods. BABEL achieved the highest F1 score in this unbalanced dataset, meaning that it found as many vulnerabilities as possible while striking a balance between false positives and false negatives. We argue that this is because BABEL preserves program dependency graph information through heuristic-based program dependency analysis of the code compared to token-based approaches. These sequence models, on the other hand, must linearize the code structure as an ordered sequence of tokens, thus ignoring internal program dependencies information such as control flow and data flow., etc. Due to the lack of structured information, sufficient vulnerability features cannot be obtained, resulting in poor predictive performance. Finally, the experimental results also show that BABEL improves the Acc and F1 scores by 1.85%, 1.67%, 2.23%, and 1%, 26.39%, 0.63%, respectively, when compared to the recent graph-based SOTA models LineVD [36], ReGVD [25], and CSGVD [37]. We hypothesize that this is because BABEL not only learns program dependencies through graph neural networks but also learns structured representations of code through hierarchical attention networks to obtain deep code understanding.

In the evaluation of CodeXGLUE, our model BABEL is notable for its efficiency, ranking third with a 65.06% accuracy despite utilizing a modestly sized dataset of 8.5M and a 21M parameter(Table III). This demonstrates that our hierarchical and graph-based approach effectively leverages syntactic and semantic information within a streamlined architecture. When comparing BABEL to CoTexT and C-BERT [41], BABEL shows competitive performance despite using significantly less data and a smaller model. CoTexT uses a dataset that is approximately 44 times larger and a model that is over 10 times the parameter size of BABEL. Similarly, C-BERT has a parameter size that is over 5 times larger than BABEL. This implies that the reduced parameter size of the BABEL does contribute to a decrease in resource usage, particularly in terms of memory and storage requirements. These considerations are crucial in deployment situations. Additionally, despite BABEL's use of CodeBERT [32] embeddings, which are not trained within our system but contribute to model performance, our approach maintains a balance between performance and resource efficiency. This aspect highlights the effectiveness of BABEL in achieving high performance with constrained resources compared to data and parameter-intensive counterparts like CoTexT [39].

B. RQ2:What is the detection performance of BABEL when detecting across different program language vulnerabilities?

To verify the vulnerability detection capability of BABEL in different programming language scenarios, we chose the FUNDED dataset [13] containing JAVA and PHP programming languages for our experiments. We use ReGVD [25], CodeBERT [32], and GraphCodeBERTGraphCodeBERT [38] as the benchmark models, specifically, we use the dataset CodeXGLUE [28] (containing only C and C++ program language) as the training set, and then the datasets of JAVA and PHP from FUNDED as the test set.

In Figure.5 and Figure.6, BABEL demonstrates strong performance in adapting to both JAVA and PHP codebases with high effectiveness. In JAVA detection, BABEL achieved the highest F1 score among benchmarked models (ReGVD, CodeBERT, and GraphCodeBERT), effectively balancing precision and recall. This performance underscores the strength of our language-independent parsing strategy, facilitating effective vulnerability detection across languages.For PHP detection, BABEL exhibited exceptionally high recall, leading to the highest F1 score. This prioritization of recall makes BABEL particularly valuable in scenarios where missing a vulnerability is more costly than false positives. BABEL's consistent performance across different programming languages supports the efficacy of our graph neural network approach, built on

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

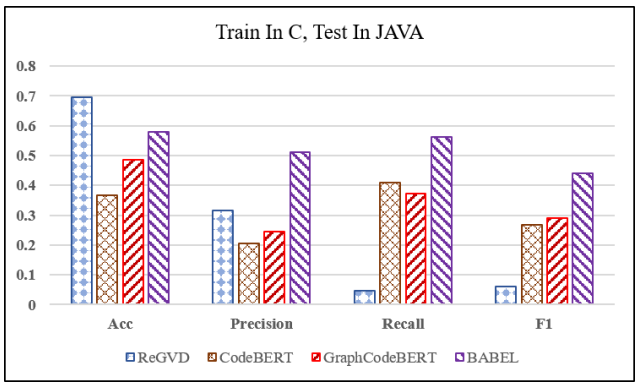


Fig. 5. Across Program Language Vulnerability Detection: Train in C, Test in JAVA.

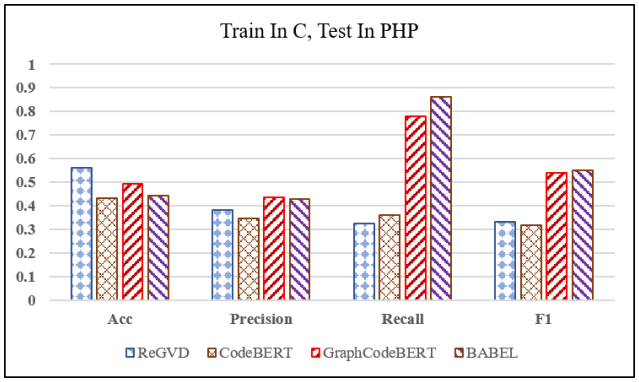


Fig. 6. Across Program Language Vulnerability Detection: Train in C, Test in PHP.

TABLE IV
COMPARISON OF F1 SCORES (IN %) FOR DIFFERENT MODELS ON VARIOUS COMMON WEAKNESS ENUMERATION (CWE) CATEGORIES WHEN TRAINED IN C, TESTED IN JAVA

CWE Type	BABEL	ReGVD	CodeBERT	GraphCodeBERT
CWE-074	**41.64**	4.40	29.37	26.86
CWE-077	**47.98**	15.53	28.43	21.94
CWE-079	**60.98**	0.14	31.97	28.04
CWE-089	**35.44**	0.00	31.18	27.29
CWE-090	**48.91**	0.00	28.38	26.75
CWE-091	32.22	37.87	25.90	**37.39**
CWE-190	35.77	0.42	28.79	**41.12**
CWE-191	35.99	0.69	27.62	**37.60**
CWE-369	**35.61**	0.00	22.48	30.62
CWE-400	35.83	0.48	20.68	**39.41**
CWE-404	**51.61**	1.49	15.71	11.38
CWE-573	**54.29**	1.65	20.79	16.48
CWE-601	**47.86**	0.00	23.06	19.35
CWE-665	38.44	0.00	26.19	**48.99**
CWE-668	**51.60**	3.00	36.53	26.69
CWE-704	**46.00**	0.00	29.49	30.91
Average	**44.01**	4.78	26.76	28.99

TABLE V
COMPARISON OF F1 SCORES (IN %) FOR DIFFERENT MODELS ON VARIOUS COMMON WEAKNESS ENUMERATION (CWE) CATEGORIES WHEN TRAINED IN C, TESTED IN PHP

CWE Type	BABEL	ReGVD	CodeBERT	GraphCodeBERT
CWE-074	75.81	48.53	51.43	**75.99**
CWE-078	**35.65**	14.54	14.32	35.17
CWE-079	54.06	31.72	32.70	**56.76**
CWE-089	**52.71**	2.10	24.18	38.77
CWE-091	34.24	37.87	26.81	**41.09**
CWE-601	**66.37**	33.42	15.64	63.67
Average	**53.14**	26.57	27.51	51.91

language-independent source code parsing. We argue that BABEL's heuristic program dependency graph construction focuses on generic programming constructs such as variables, functions, and control flow statements common to C, JAVA, and PHP, despite their unique syntactic and semantic features. This technique enables BABEL to universally capture and utilize structural and semantic code features, regardless of syntactic differences between languages.

C. RQ3:Does BABEL understand the nature of vulnerabilities in different languages?

To further verify BABEL's vulnerability detection capabilities in different programming language scenarios in the real world, we verified the effectiveness of BABEL on different Common Weakness Enumerations (CWEs) [46]. The CWE provides a comprehensive list of vulnerability types, serving as a standard terminology for identifying and describing vulnerabilities. Specifically, we train on CodeXGLUE [28] and test on different CWEs for JAVA and PHP provided in the FUNDED [13] dataset. As shown in TableIV and TableV, our BABEL model stands out with F1 scores of 44.01% and 53.14% on the JAVA and PHP datasets, respectively, which are significantly higher than the average, and an improvement of 15.02% and 1.23% over the previous best benchmark. This suggests that our approach generalizes well to different programming languages, implying better vulnerability detection capabilities.

Taking the JAVA dataset, which has a wider variety of vulnerabilities, as an example, ReGVD [25] performed less effectively across all CWE categories, with an average F1 score of only 4.78%. This suggests its limited cross-language generalization capability, as it constructs graphs treating the text as natural language sequences, leading to a weaker understanding of semantics and syntactic features related to vulnerability detection in languages other than C. GraphCodeBERT [38] and CodeBERT [32] performed moderately, with an average F1 score of 28.99% and 26.76%. Considering that both GraphCodeBERT and CodeBERT are designed to understand both programming and natural languages, it has some cross-language generalization ability, but GraphCodeBERT learns vector representations of source code based on data flow graph, which results in higher F1 scores than CodeBERT.

In terms of individual vulnerability types, BABEL surpassed baseline models by 35.9% and 15.3% in CWE-404 (Resource Management) [47] and CWE-668 (Improper Access Control) [48] vulnerabilities, respectively, indicating its strong control

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
RESULTS OF THE ABLATION STUDY OF CODEXGLUE. BOLD TEXT INDICATES OUR FULL MODEL." DDG" AND "CDG" DENOTE DATA DEPENDENCY GRAPHS AND CONTROL DEPENDENCY GRAPHS.

	ACC	Precision	Recall	F1
w/o DDG	63.58	67.24	40.4	50.47
w/o CDG	63.25	68.16	37.53	48.41
w/o GNN	62.99	66.71	38.8	49.07
BABEL	**65.08**	**71.41**	**40.00**	**51.28**

flow analysis capability. Such vulnerabilities require the model to understand the program's execution paths, which rely less on specific language syntax and more on understanding the program's logical flow. Meanwhile, BABEL achieved the highest F1 scores in identifying vulnerabilities related to improper data handling (e.g., CWE-079 [49], CWE-089 [50], CWE-091 [51]), demonstrating its robust data flow analysis capability, as data handling constructs can vary significantly across languages. BABEL's performance in CWE-190 (Integer Overflow or Wraparound) [52] and CWE-191 (Integer Underflow) [53] was competitive, with F1 scores of 35.77% and 35.99%, respectively. Although effective, BABEL did not significantly outperform the baseline models in these cases. We believe this is due to the nature of numerical errors, which may be easier to detect and less affected by language-specific nuances. Similarly, our model shows similar semantic understanding capabilities on the PHP dataset, which demonstrates BABEL's knowledge transfer capabilities.

D. RQ4: How does HAN, Data-dependency, and Control-dependency contribute to the performance of BABEL?

To validate the impact of the different components, we conducted experiments on several ablation modules, including: (1) BABEL w/o DDG: This variant operates without the Data Dependency Graph to evaluate the significance of data flow relationships. (2) BABEL w/o CDG: By removing the Control Dependency Graph, we analyzed the impact of control flow information. (3) BABEL w/o GNN: This version employs only the Hierarchical Attention Network to process the structural aspects of code, leaving out any inter-line dependencies.

Comparative performance metrics are described in Table 4 in detail. The performance of both BABEL (without CDG) and BABEL (without DDG) is degraded due to the lack of program semantic knowledge.

Specifically, excluding DDG leads to a 1.5% decrease in accuracy, and lack of CDG leads to a 1.83% decrease in accuracy. Removing the Data Dependency Graph (DDG) results in a significant drop in recall while maintaining moderate precision. This indicates that the DDG is essential for accurately identifying true positive cases (vulnerabilities). It likely aids in comprehending the usage and flow of data within the code, which is crucial for detecting specific types of vulnerabilities. On the other hand, the removal of the Control Dependency Graph (CDG) leads to the lowest recall among all variants, underscoring its critical role in tracing the control structures within the code.

When removing all program dependency graphs (BABEL w/o GNN), the performance of the model decreases significantly, with the lowest performance compared to other ablation modules, and the accuracy, precision, recall, and F1 value on the CodeXGLUE dataset decrease by 2.09%, 4.70%, 1.20% and 2.21% on the CodeXGLUE dataset, respectively. This result indicates that the program dependency graphs we constructed all play a key role in BABEL. This highlights the critical nature of the program dependency graphs; the DDG emphasizes variable usage while the CDG illuminates the code's execution path. Their synergistic interaction significantly enhances the model's ability to characterize code semantics effectively.

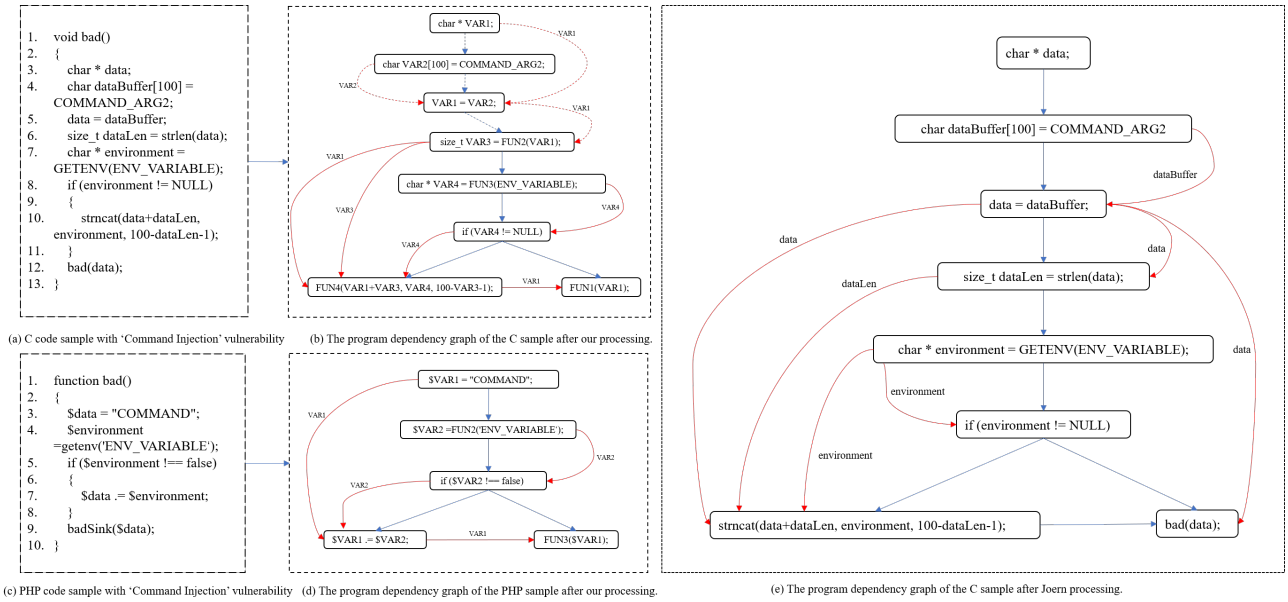
V. DISCUSSION

A. Case Study

In this section, we present a case study to demonstrate the performance of our method in capturing the internal structure of the code as well as cross-programming language independence. In this case, the OS Command Injection vulnerability in C code and PHP code stems from similar data flow and control flow patterns. Our model successfully captures the error paths resulting from the combined effects of such data dependencies and control dependencies.

Specifically, in Figure 7, line 7 in (a) and line 4 in (c) are vulnerable statements, i.e., the C code: "GETENV(ENV_VARIABLE)" gets data from an environment variable, while "getenv('ENV_VARIABLE')" in PHP code performs the same operation. By performing variable symbolization, our model assumes that this step introduces external inputs into data dependencies (corresponding to the solid red line connecting the two lines). During program execution, the values of the environment variables are appended directly to the command strings via 'if' conditional statements, which are linked together in our control dependency graph (corresponding to the blue solid line connecting to the 'if' statement). This shows that our approach constructs data dependencies without relying on a parser. Also, as shown in Figure 7 (e), we have parsed this code through the most dominant compiler available today: Joern [23]. After comparison, we can find that our data dependency graph is highly similar to the one parsed by the parser, which proves the rationality of our approach. Specifically, our program dependency graph connects more edges because we capture all usage of variables. We believe that different programming languages with similar data and control flows may lead to the same vulnerability and that this pattern is language-independent and is a common vulnerability in software that interacts with system-level functionality. This example also illustrates that our approach simultaneously satisfies the need to generate program dependency graphs without relying on a full-fledged parser, and is also able to capture vulnerabilities in a language-independent manner.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.



(a) C code sample with 'Command Injection' vulnerability (b) The program dependency graph of the C sample after our processing.

(c) PHP code sample with 'Command Injection' vulnerability (d) The program dependency graph of the PHP sample after our processing.

(e) The program dependency graph of the C sample after Joern processing.

Fig. 7. An example to illustrate the applicability of our composition rules to vulnerability detection in different programming languages. The red arrows represent data dependencies and the blue arrows represent control dependencies. both the C code (a) and the PHP code (c) contain a "command injection" vulnerability. Although the programs are written in two different languages, they have similar control and data dependency leading to a shared buggy path (which we denote by the solid line).

B. Threat to Validity

Current vulnerability detection tasks mainly focus on C or Cpp language and lack high-quality benchmark data sets for other programming languages (such as JAVA and Swift). This scarcity hinders the ability to perform a more comprehensive evaluation of general vulnerability detection methods across different programming environments. At the same time, it also limits the model's ability without training on a large scale of vulnerability detection corpora. This limitation affects models that require diverse and representative samples to generalize well across different scenarios.

VI. Realted Work

Existing techniques can be divided into three groups based on how source code is represented and the type of learning model used: token-based methods, graph-based methods, and pre-trained language model-based methods.

Token-based methods. Token-based methods [7]–[9], [18], [54], [55] treat code as a flat text sequence and utilize techniques from the natural language processing domain to process it. Early work by Russell et al. [7] integrated deep learning with natural language processing techniques for source code vulnerability detection. They modeled code at the function level, used CNN [26] to extract code features, and captured key information. Li et al. proposed VulDeePecker [9], which extracts code snippets from source code by slicing the program, transforms them into corresponding vector representations, and inputs them into a Bi-LSTM [27] model for vulnerability detection. They further introduced muVulDeePecker [8], updating the program processing approach from VulDeePecker [9] to accommodate data and control dependency information in the code, and added code attention to detect multiple vulnerabilities.

Graph-based methods. Compared with token-based methods, researchers have designed various Graph Neural Network (GNN) models [56]–[58] to better utilize the structural information of code. Several methods [11], [20], [59], [60] abstract the code into graphs (e.g., AST, CFG, DFG, PDG) and use these graphs as model input. For instance, Code2Seq [6] utilizes AST and path context features to encode source code as sequence input for training deep neural network models. Similarly, Devign [19] represents source code fragments as code attribute graphs derived from static analysis tools [23], applying GNNs to learn structural information for vulnerability detection. In another approach, Reveal [19] constructs a graph structure representing the program's dependency information, then uses GGNN [57] to update node features, thereby learning both code functionality and structure. IVDetect [43] employs a Graph Convolutional Network (GCN) with a context vector to predict vulnerabilities. Furthermore, ReGVD [25] detects vulnerabilities by building graph relationships between tokens using a sliding window mechanism. CSGVD [37] employs an M-BFA attention pool to aggregate node information for graph feature representation.

Pre-trained language model-based methods. Due to the remarkable potential demonstrated by pre-trained language models [32], [38], [42], [44], [61]–[63], many researchers have started to pre-train models on code for vulnerability detection. For example, CodeBERT [32], as one of the earliest pre-training architectures, has been applied to downstream tasks

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

like function-level defect detection. Furthermore, to study and learn representations based on large-scale graphs, GraphCodeBERT [38] merges graph structures into Transformer-based models and learns vector representations of source code based on data flow. VulBERTa [44] utilizes a language-aware tokenization pipeline that enhances BPE tokenization by incorporating custom pre-defined tokens like AST nodes to generate better tokenization that preserves syntactic structure. LineVul [42] employed a transformer model pre-trained on a vast collection of diverse open-source projects.

Although token-based approaches are highly adaptable, they face challenges in capturing deeper levels of context and structure. BABEL addresses this limitation by modeling structural and semantic features of source code using its hierarchical structure and heuristic graph construction techniques. In addition, BABEL constructs program dependency graphs with minimal syntactic assumptions about the code, thereby mitigating the reliance of existing graph methods on parsers for program dependency representations. This results in a more comprehensive and adaptable model for vulnerability detection across all programming languages. By integrating both graph-based and token-based approaches, BABEL enhances the model's code comprehension while reducing its parameters, making it more suitable for practical applications.

VII. Conclusion

In this work, we present a novel framework for detecting software vulnerabilities in a language-independent manner. Our model automatically encodes the source code, eliminating the need for a code parser, and adapts to various programming languages with high generality and scalability. We conduct comprehensive experiments on public datasets, and the experimental results demonstrate the effectiveness of our model and its potential for adaptation in diverse software development environments. In the future, we will explore automated modeling the structural features of code and the grammatical assumptions inherent in code. Additionally, we plan to extend our approach to address different types of vulnerabilities and coding patterns.

Acknowledgments

This work is supported by the National Natural Science Foundation of China [grant number: 61976062].

References

[1] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, "Bingo: Cross-architecture cross-os binary search," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 678–689.

[2] Z. Xu, B. Chen, M. Chandramohan, Y. Liu, and F. Song, "Spain: security patch analysis for binaries towards understanding the pain and pills," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 2017, pp. 462–472.

[3] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, "Predicting vulnerable software components," in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 529–540.

[4] V. H. Nguyen and L. M. S. Tran, "Predicting vulnerable software components with dependency graphs," in *Proceedings of the 6th International Workshop on Security Measurements and Metrics*, 2010, pp. 1–8.

[5] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, "Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities," *IEEE transactions on software engineering*, vol. 37, no. 6, pp. 772–787, 2010.

[6] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating sequences from structured representations of code," in *International Conference on Learning Representations*, 2018.

[7] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018, pp. 757–762.

[8] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2019.

[9] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*.

[10] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Transactions on Software Engineering*, 2021.

[11] X.-C. Wen, C. Gao, J. Ye, Z. Tian, Y. Jia, and X. Wang, "Meta-path based attentional graph learning model for vulnerability detection," *arXiv preprint arXiv:2212.14274*, 2022.

[12] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, and Y. Wu, "Vulsniper: Focus your attention to shoot fine-grained vulnerabilities," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, ser. IJCAI'19. AAAI Press, 2019, p. 4665–4671.

[13] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1943–1958, 2020.

[14] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2365–2376.

[15] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 292–303.

[16] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1746–1751. [Online]. Available: https://aclanthology.org/D14-1181

[17] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–29, 2019.

[18] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2021.

[19] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[20] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J. M. Zhang, and Q. Liao, "Vulnerability detection with graph simplification and enhanced graph representation learning," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 2275–2286. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00191

[21] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, apr 2021. [Online]. Available: https://doi.org/10.1145/3436877

[22] X. Cheng, G. Zhang, H. Wang, and Y. Sui, "Path-sensitive code embedding via contrastive learning for software vulnerability detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022. New York, NY, USA:

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

Association for Computing Machinery, 2022, p. 519–531. [Online]. Available: https://doi.org/10.1145/3533767.3534371

[23] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 590–604.

[24] B. Yuan, Y. Lu, Y. Fang, Y. Wu, D. Zou, Z. Li, Z. Li, and H. Jin, "Enhancing deep learning-based vulnerability detection by building behavior graph model," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2262–2274.

[25] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "Regvd: Revisiting graph neural networks for vulnerability detection," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 178–182.

[26] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

[27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, pp. 1735–80, 12 1997.

[28] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.

[29] F. Dong, Y. Zhang, and J. Yang, "Attention-based recurrent convolutional neural network for automatic essay scoring," in *Proceedings of the 21st conference on computational natural language learning (CoNLL 2017)*, 2017, pp. 153–162.

[30] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[31] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.

[32] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.139

[33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[34] D. A. Wheeler. Flawfinder. [Online]. Available: https://www.dwheeler.com/flawfinder/

[35] Cppcheck. Cppcheckr. [Online]. Available: http://cppcheck.sourceforge.net/

[36] D. Hin, A. Kan, H. Chen, and M. A. Babar, "Linevd: Statement-level vulnerability detection using graph neural networks," in *Proceedings of the 19th international conference on mining software repositories*, 2022, pp. 596–607.

[37] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "Csgvd: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *Journal of Systems and Software*, vol. 199, p. 111623, 2023.

[38] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, L. Shujie, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," in *International Conference on Learning Representations*, 2020.

[39] L. Phan, H. Tran, D. Le, H. Nguyen, J. Annibal, A. Peltekian, and Y. Ye, "Cotext: Multi-task learning with code-text transformer," in *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021)*, 2021, pp. 40–47.

[40] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 2655–2668. [Online]. Available: https://aclanthology.org/2021.naacl-main.211

[41] X. Wu, S. Lv, L. Zang, J. Han, and S. Hu, "Conditional bert contextual augmentation," in *Computational Science–ICCS 2019: 19th International Conference, Faro, Portugal, June 12–14, 2019, Proceedings, Part IV 19*. Springer, 2019, pp. 84–95.

[42] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.

[43] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 292–303. [Online]. Available: https://doi.org/10.1145/3468264.3468597

[44] H. Hanif and S. Maffeis, "Vulberta: Simplified source code pre-training for vulnerability detection," in *2022 International joint conference on neural networks (IJCNN)*. IEEE, 2022, pp. 1–8.

[45] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[46] CWE. Common weakness enumeratio. [Online]. Available: https://cwe.mitre.org/

[47] Cwe-404: Improper resource shutdown or release. [Online]. Available: https://cwe.mitre.org/data/definitions/404.html/

[48] Cwe-668: Exposure of resource to wrong sphere. [Online]. Available: https://cwe.mitre.org/data/definitions/668.html

[49] Cwe-79: Improper neutralization of input during web page generation ('cross-site scripting'). [Online]. Available: https://cwe.mitre.org/data/definitions/79.html

[50] Cwe-89: Improper neutralization of special elements used in an sql command ('sql injection'). [Online]. Available: https://cwe.mitre.org/data/definitions/89.html

[51] Cwe-91: Xml injection (aka blind xpath injection). [Online]. Available: https://cwe.mitre.org/data/definitions/91.html

[52] Cwe-190: Integer overflow or wraparound. [Online]. Available: https://cwe.mitre.org/data/definitions/190.html

[53] Cwe-191: Integer underflow (wrap or wraparound). [Online]. Available: https://cwe.mitre.org/data/definitions/191.html

[54] V. Nguyen, T. Le, O. De Vel, P. Montague, J. Grundy, and D. Phung, "Information-theoretic source code vulnerability highlighting," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.

[55] Z. Tang, Q. Hu, Y. Hu, W. Kuang, and J. Chen, "Sevuldet: A semantics-enhanced learnable vulnerability detector," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2022, pp. 150–162.

[56] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

[57] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated graph sequence neural networks," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016. [Online]. Available: http://arxiv.org/abs/1511.05493

[58] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Semantic Web*, A. Gangemi, R. Navigli, M.-E. Vidal, P. Hitzler, R. Troncy, L. Hollink, A. Tordai, and M. Alam, Eds. Cham: Springer International Publishing, 2018, pp. 593–607.

[59] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "Mvd: memory-related vulnerability detection based on flow-sensitive graph neural networks," ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1456–1468. [Online]. Available: https://doi.org/10.1145/3510003.3510219

[60] X. Cheng, G. Zhang, H. Wang, and Y. Sui, "Path-sensitive code embedding via contrastive learning for software vulnerability detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 519–531. [Online]. Available: https://doi.org/10.1145/3533767.3534371

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.

[61] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://doi.org/10.18653/v1/n19-1423

[62] S. Gao, C. Gao, C. Wang, J. Sun, D. Lo, and Y. Yu, "Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1933–1945.

[63] Y. Ding, L. Buratti, S. Pujar, A. Morari, B. Ray, and S. Chakraborty, "Towards learning (dis)-similarity of source code from program contrasts," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 6300–6312. [Online]. Available: https://aclanthology.org/2022.acl-long.436

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 24,2024 at 02:20:03 UTC from IEEE Xplore. Restrictions apply.